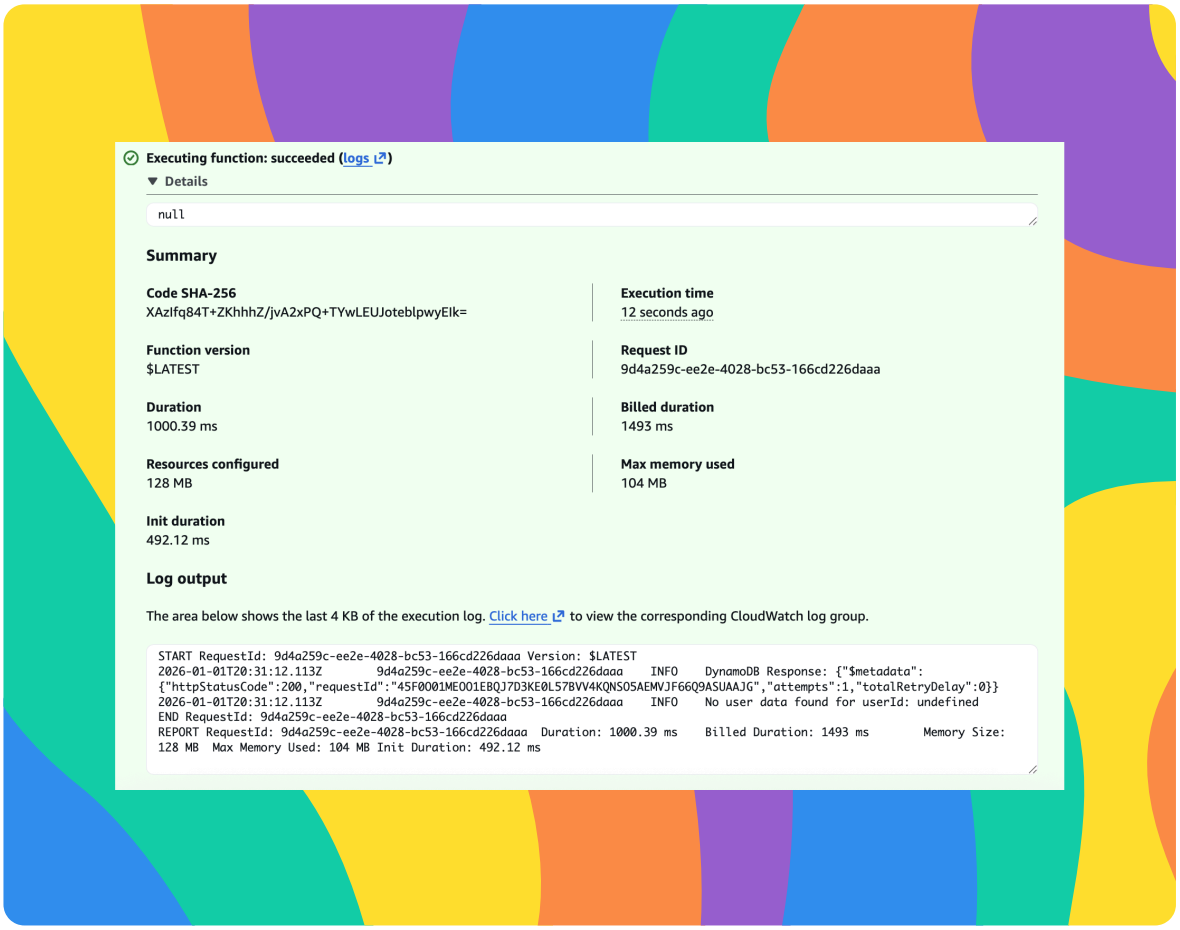
nextwork.org

Fetch Data with AWS Lambda

CH Arpita Chowdhury



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Introducing Today's Project!

In this project, I will demonstrate how to build a secure, serverless data retrieval workflow using AWS, including creating a DynamoDB table, accessing it with a Lambda function, testing the function, and applying proper IAM permissions and inline policies. I’m doing this project to learn how real-world serverless applications securely store and retrieve user data, and how permissions, testing, and least-privilege access are applied in production-ready cloud architectures.

Tools and concepts

Services I used were Amazon DynamoDB, AWS Lambda, AWS IAM, and Amazon CloudWatch. Key concepts I learnt include Lambda functions, execution roles and IAM policies, least-privilege security using inline policies, how Lambda interacts with DynamoDB using the AWS SDK, handling permission errors like AccessDeniedException, and validating serverless applications through testing and CloudWatch logs.

Project reflection

This project took me approximately 2 hours to complete. The most challenging part was debugging IAM permission issues, especially identifying why the Lambda function could execute successfully but still fail due to missing DynamoDB permissions. It was most rewarding to see the Lambda function successfully retrieve data from DynamoDB after applying the correct least-privilege inline policy, because it confirmed my understanding of serverless security, permissions, and real-world AWS troubleshooting.

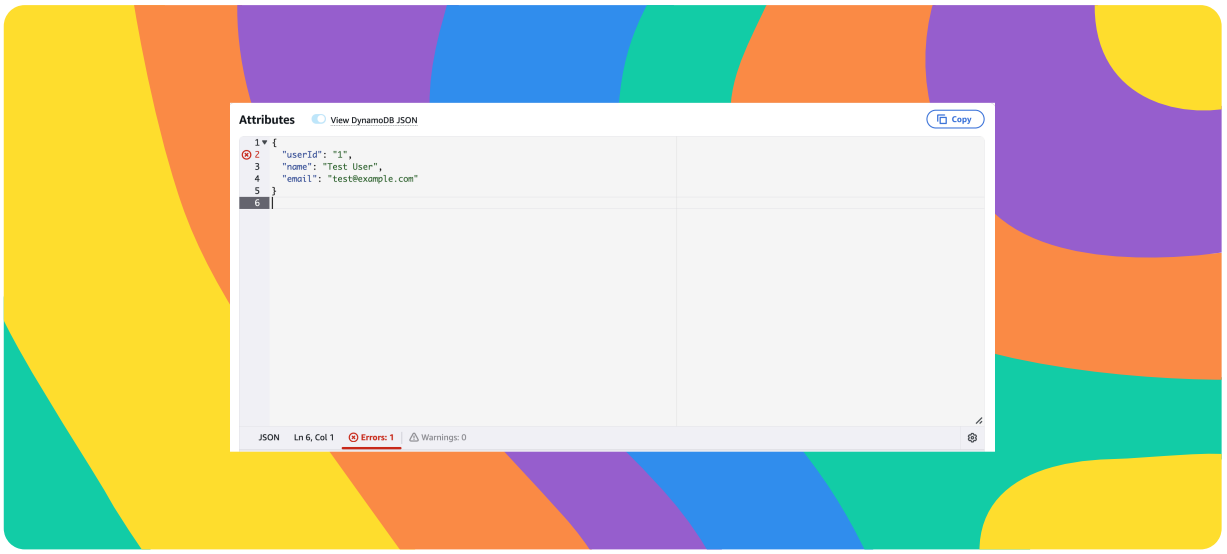
I chose to do this project today because I wanted hands-on practice with real AWS services and to strengthen my understanding of serverless architectures using Lambda, DynamoDB, and IAM, which are core skills for cloud and backend roles. Working through the project helped me connect theory with practical troubleshooting, especially around permissions and security best practices.

CH
Arpita Chowdhury
NextWork Student

Project Setup

To set up my project, I created a database using Amazon DynamoDB. The partition key is userId (string), which means each item in the table is uniquely identified and accessed using a user's ID, allowing DynamoDB to efficiently store and retrieve user-specific data.

In my DynamoDB table, I added a sample user item with attributes such as userId, name, and email. DynamoDB is schemaless, which means I don't have to define all attributes in advance each item can have different attributes as long as it includes the required partition key.



AWS Lambda

AWS Lambda is a serverless compute service that runs your code in response to events without you needing to provision or manage servers. I'm using Lambda in this project to retrieve user data from my DynamoDB table (using the userId partition key) and return it on demand, so I can test and validate database access through a secure serverless function.

CH
Arpita Chowdhury
NextWork Student
nextwork.org

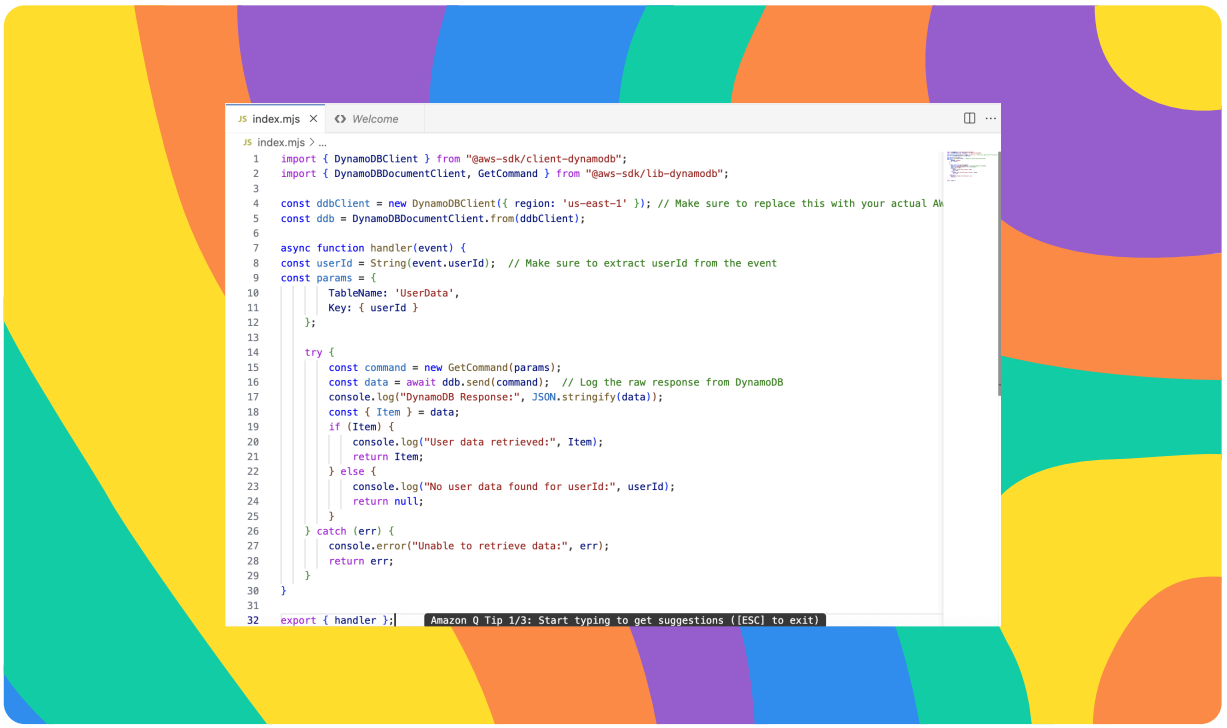
AWS Lambda Function

My Lambda function has an execution role, which is an IAM role that defines what AWS services and resources the function is allowed to access. By default, the role grants basic permissions to write logs to Amazon CloudWatch, so the Lambda function can record execution details and error messages while running.

My Lambda function will take a userId from the incoming event, query the UserData DynamoDB table for that user, and return the corresponding user record if it exists, while gracefully handling errors and logging helpful messages if the data can’t be retrieved.

The code uses the AWS SDK, which is a collection of libraries that let developers interact with AWS services programmatically using code instead of the AWS Console. My code uses the SDK to connect to DynamoDB, send a GetItem request using the userId, and retrieve user data securely from the UserData table within my Lambda function.

Arpita Chowdhury
NextWork Student



CH
Arpita Chowdhury
NextWork Student

Function Testing

To test whether my Lambda function works, I created and ran a test event in the AWS Lambda console that passes a sample userId to the function. The test is written in JSON, which simulates the input event Lambda would receive from an application. If the test is successful, I'd see a successful execution result along with the expected user data returned from DynamoDB and no permission or runtime errors in the logs.

The test displayed a "success" because AWS Lambda successfully invoked and ran my function (the execution completed and returned a response), but the function's response was actually an AccessDeniedException because the Lambda execution role (RetrieveUserData-role-...) does not have an identity-based IAM policy that allows dynamodb:GetItem on the UserData table, so DynamoDB denied the request.

Arpita Chowdhury
NextWork Student

CH
Arpita Chowdhury
NextWork Student
nextwork.org

Function Permissions

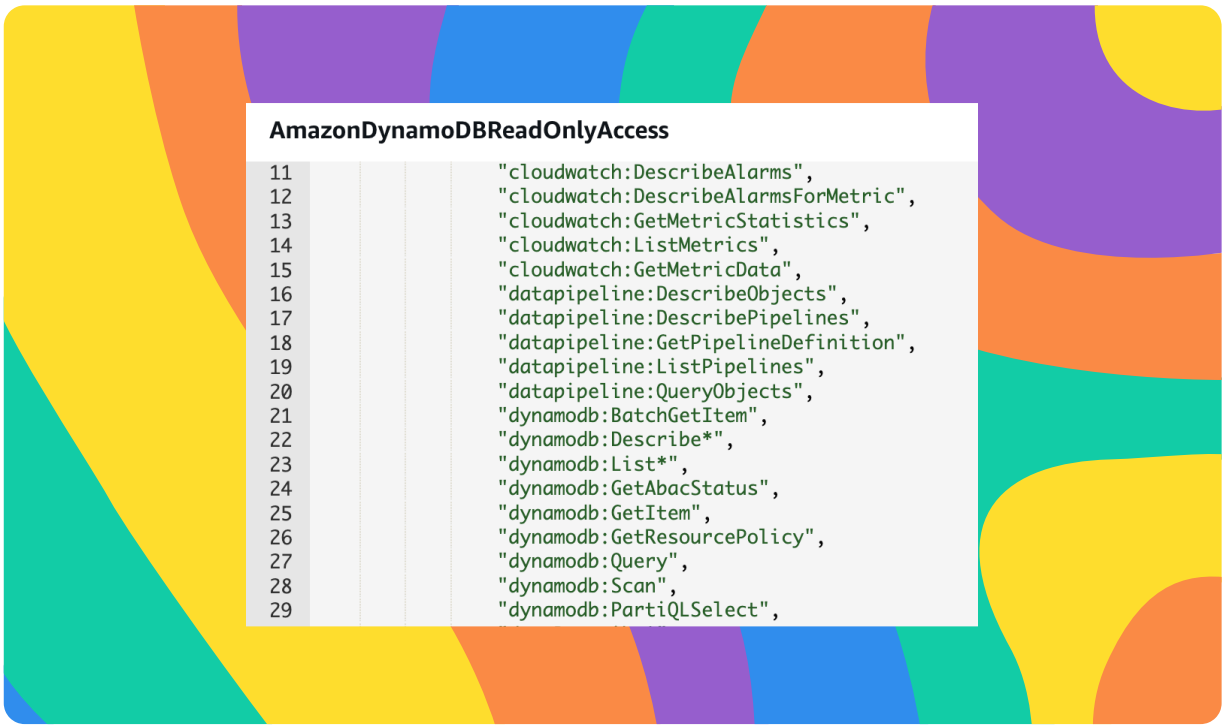
To resolve the AccessDenied error, I attached a DynamoDB read permission policy (so my Lambda execution role can perform dynamodb:GetItem on the UserData table), because the error message showed that the function was being blocked specifically for the GetItem action due to missing IAM permissions.

There were four DynamoDB permission policies I could choose from, but I didn't pick AWSLambdaDynamoDBExecutionRole or AWSLambdaInvocation-DynamoDB, because both of these are designed for DynamoDB streams and event-driven triggers, not for simply reading data from a DynamoDB table. My Lambda function's job is only to retrieve an item using GetItem, not to listen to table changes or react to stream events, so those policies would have given the wrong type of access and solved a different problem than the one shown in the error message.

I picked AmazonDynamoDBReadOnlyAccess because my error message showed the exact denied action was dynamodb:GetItem, and this policy includes GetItem (plus other read operations like Query, Scan, and Describe), which is exactly what my Lambda function needs to retrieve user data without modifying anything. I also didn't pick AmazonDynamoDBFullAccess because it grants write and admin permissions (like creating/deleting tables and writing/deleting items), which my function does not need. Giving extra permissions violates least privilege and makes the system less secure if the function or role is ever misused. So, AmazonDynamoDBReadOnlyAccess was the right choice because it fixes the AccessDenied issue by allowing read actions such as GetItem while keeping my Lambda function restricted from making changes to the database.

CH

Arpita Chowdhury
NextWork Student



CH
Arpita Chowdhury
NextWork Student

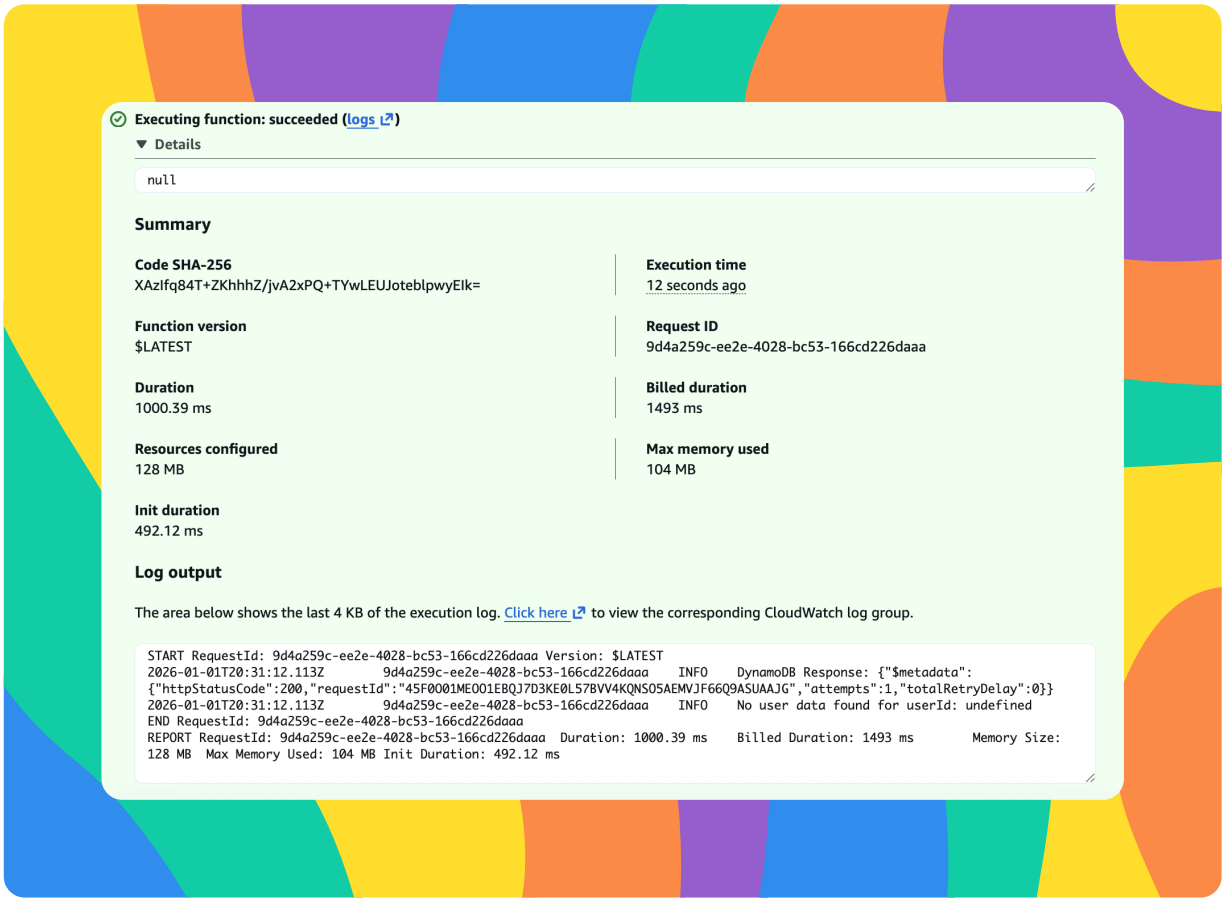
Final Testing and Reflection

To validate my new permission settings, I re-ran the Lambda function test with the same test event after attaching the AmazonDynamoDBReadOnlyAccess policy to the execution role. The results were successful execution with no AccessDenied error, because the Lambda function now has permission to call dynamodb:GetItem. The response returned null specifically because the test event did not pass a valid userId value (it was undefined), so DynamoDB correctly found no matching item, confirming that permissions are fixed and the function logic is working as expected.

Web apps are a popular use case of using Lambda and DynamoDB. For example, I could build a serverless backend for a web application where Lambda handles API requests (such as fetching user profiles or account details) and DynamoDB stores the user data, allowing the app to scale automatically, respond quickly to users, and avoid managing any servers or databases manually.

CH

Arpita Chowdhury
NextWork Student



CH
Arpita Chowdhury
NextWork Student
nextwork.org

Enahancing Security

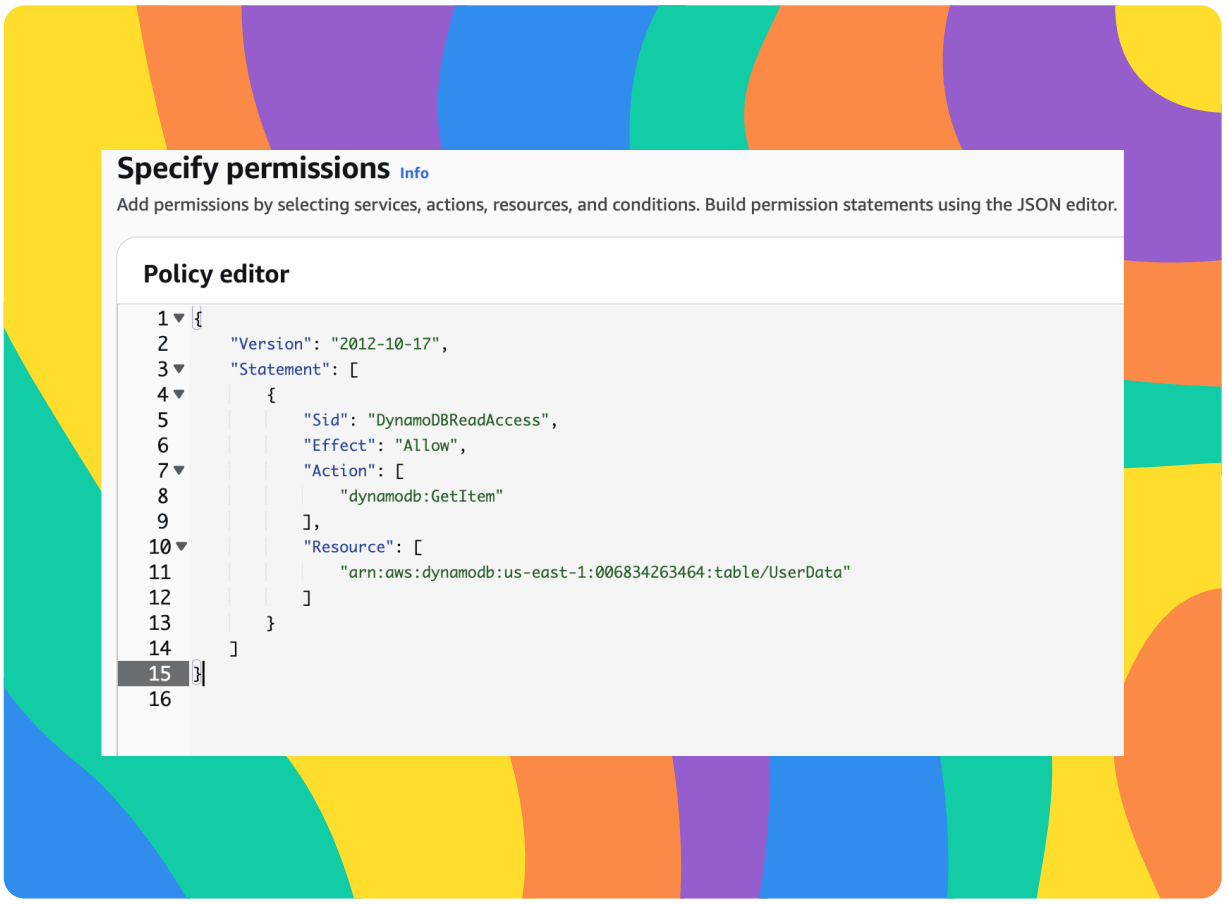
For my project extension, I challenged myself to replace the broad AmazonDynamoDBReadOnlyAccess managed policy with a tightly scoped inline IAM policy that only allows the exact read actions my Lambda needs (like dynamodb:GetItem) on the specific UserData table. This will follow the principle of least privilege, reduce unnecessary permissions to other DynamoDB tables/resources, and make my serverless app more secure and production-ready while still keeping the function working.

To create the permission policy, I used the IAM JSON policy editor (inline policy editor), because it lets me write a least-privilege policy that grants my Lambda function only the exact permission it needs (dynamodb:GetItem) on the specific UserData table ARN, instead of giving read access to every DynamoDB table in my account.

When updating a Lambda function’s permission policies, you could risk breaking the function by removing required permissions, which can lead to AccessDenied errors or cause the function to fail at runtime even though the code itself is correct. I validated that my Lambda function still works by re-running the Lambda test event after applying the inline policy and confirming that it successfully retrieved data from the UserData DynamoDB table without any permission errors.

CH

Arpita Chowdhury
NextWork Student



The place to learn & showcase your skills

Check out nextwork.org for more projects